

線形回帰と正則化

`fit()` を自分の手で作る

rkskmt

ここまでの道具

- **勾配降下法**：傾きの逆方向に少しずつ進んで、損失が最小の場所を探す
- ただし前回最小化したのは、練習用の数学の関数 $f(x) = x^2 - 4x + 3$ だった

① 今回の問い

勾配降下法で、**本物のモデル**を **fit** できるか？ — **fit()** の自作、第1号

回帰問題とは

- **分類**：どのグループか答える（Salmon か Sea bass か）
- **回帰**：**数値そのもの**を予測する（重さは何グラムか）
- 概論の「AIが扱うタスク」の2番目が、今日ようやく登場

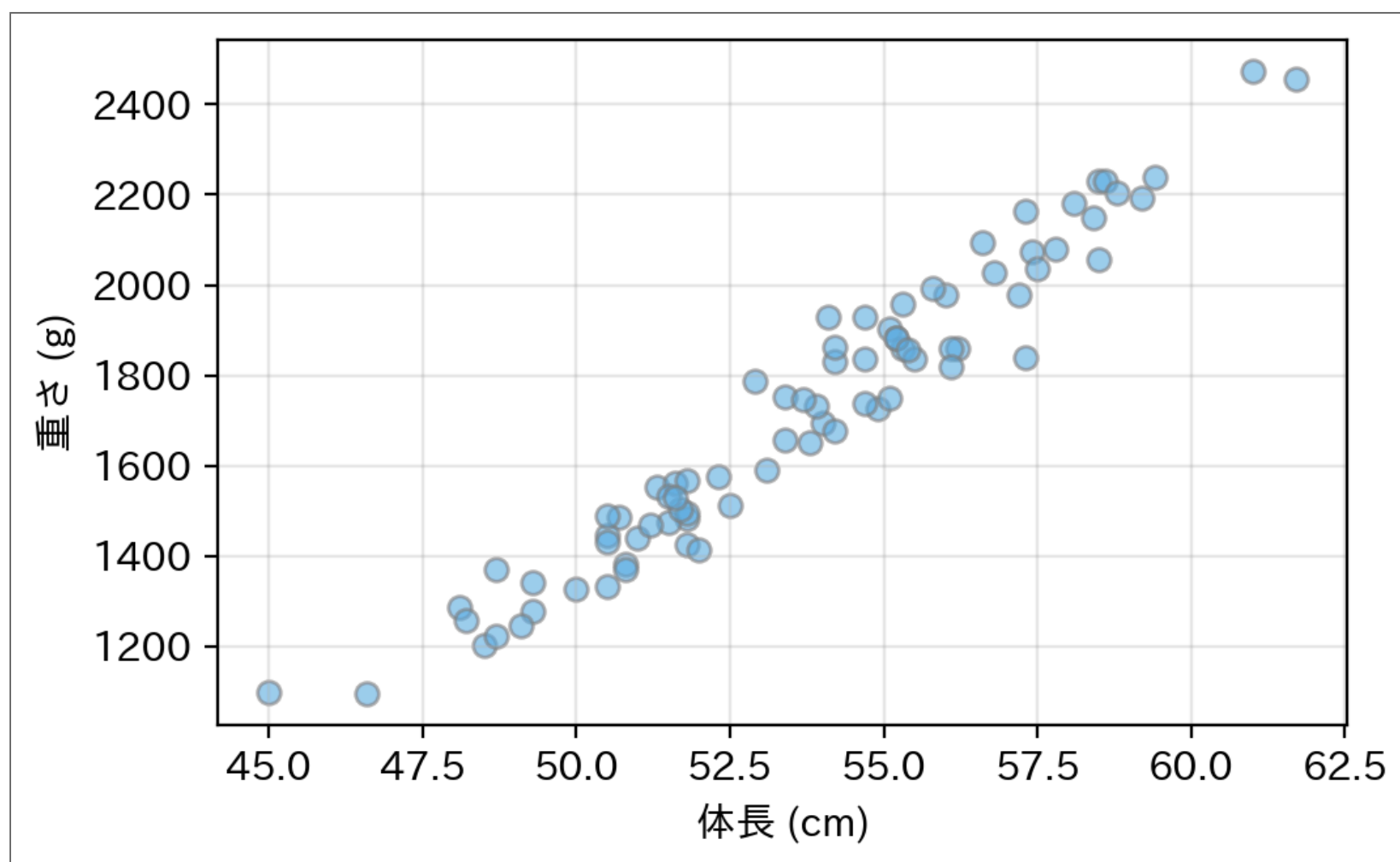
	分類	回帰
出力	グループ名	数値
例	この魚は Salmon	この魚は約 1900g



今回の問題：体長から重さを予測する

- 次元削減の回で測った**体長**を使う
- 体長を測るだけで重さが分かれば、**秤に乗せる手間が省ける**

▶ 魚データの定義（クリックして展開）



- 直線でいけそう。**直線のパラメータは傾き a と切片 b の2個**
- ではどの直線が「一番良い」？ — **良さを測る物差し**が要る

4

損失関数：二乗誤差（MSE）

- 各魚について「予測と実際のズレ」を測り、二乗して平均する

$$L(a, b) = \frac{1}{N} \sum_i \left(\underbrace{ax_i + b}_{\text{予測}} - \underbrace{t_i}_{\text{実際}} \right)^2$$

- **MSE（Mean Squared Error / 平均二乗誤差）** と呼ぶ

📢 クイズ

ズレの測り方なら**絶対値**でもいいはず。なぜ二乗？

- 絶対値は 0 のところで**微分できない**（カクッと折れている）
- 二乗なら微分がきれい
- 勾配降下法を使う前提で、損失は「微分しやすいもの」を選ぶ

5

勾配を手で出す

L を a と b でそれぞれ微分する（合成関数の微分）：

$$\frac{\partial L}{\partial a} = \frac{2}{N} \sum_i (ax_i + b - t_i) x_i \qquad \frac{\partial L}{\partial b} = \frac{2}{N} \sum_i (ax_i + b - t_i)$$

- 構造は単純：「予測と正解の差」を入力の大きさに応じて配る
- あとは勾配降下法に渡すだけ：

$$a \leftarrow a - \eta \frac{\partial L}{\partial a}, \qquad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

勾配降下法で fit する

- スケールを揃えてから（[クラスタリングの回の教訓](#)）。学習率を1つで済ませるため）

Code

```
1 x = (lengths - lengths.mean()) / lengths.std() # 標準化した体長
2 t = (weights - weights.mean()) / weights.std() # 標準化した重さ
3
4 a, b = 0.0, 0.0 # 直線はゼロからスタート
5 lr = 0.1 # 学習率
6
7 for i in range(1, 101):
8     pred = a * x + b
9     grad_a = 2 * np.mean((pred - t) * x)
10    grad_b = 2 * np.mean(pred - t)
11    a -= lr * grad_a
12    b -= lr * grad_b
13    if i in (1, 3, 10, 50, 100):
14        print(f"更新 {i:3d}回 | a={a:.4f} b={b:.4f} | MSE={np.mean((a*x+b - t)**2):.4f}")
```

Result

更新	1回		a=0.1946	b=0.0000		MSE=0.6593
更新	3回		a=0.4747	b=0.0000		MSE=0.3017
更新	10回		a=0.8684	b=0.0000		MSE=0.0645
更新	50回		a=0.9728	b=0.0000		MSE=0.0536
更新	100回		a=0.9728	b=0.0000		MSE=0.0536

- 50回あたりで動かなくなった = 収束



- さっきのコード の `lr = 0.1` を書き換えて実行

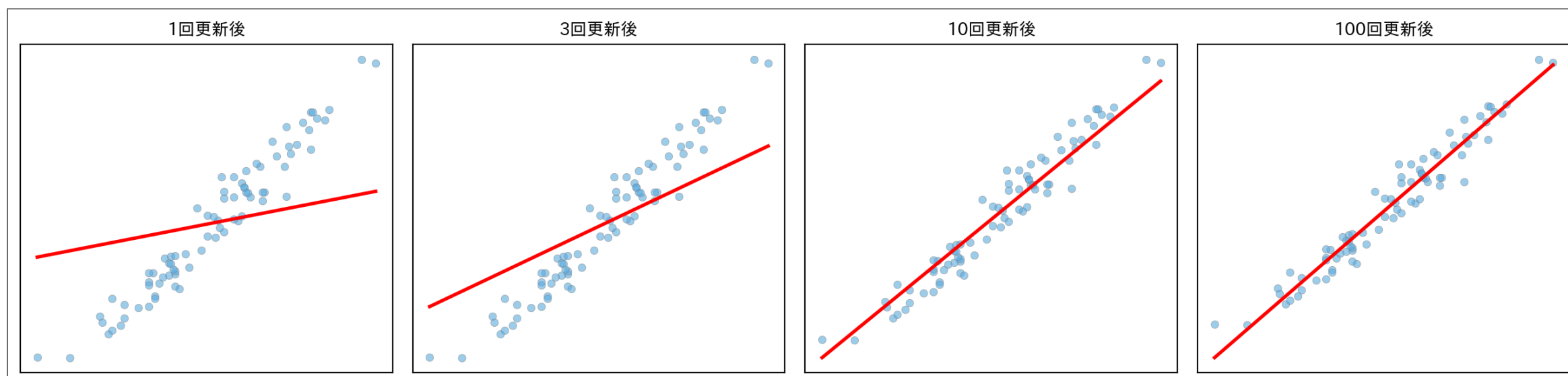
- | | | |
|---|----------------|------------------------|
| 1 | 目で見えて分かるほど遅くする | <code>lr = 0.01</code> |
| 2 | ギリギリを攻める | <code>lr = 0.9</code> |
| 3 | 壊す | <code>lr = 1.1</code> |

💡 ここで観察

- **0.9** は行きすぎて戻ってを繰り返しながら、それでも収束する
- **1.1** は MSE がどうなっていく？ — 前回の「学習率が大きすぎると発散」を、本物の `fit()` で再現できた

8

直線が合っていく様子



- 寝ていた直線が起き上がり、点の真ん中に吸い付いていく
- NN最小単位の回で見る「境界が動く」と **同じ現象の回帰版**

9

答え合わせ：sklearn と比べる

Code

```
1 from sklearn.linear_model import LinearRegression
2
3 reg = LinearRegression().fit(x.reshape(-1, 1), t)
4 print(f"自作の勾配降下: a={a:.4f}, b={b:.4f}")
5 print(f"sklearn の fit: a={reg.coef_[0]:.4f}, b={reg.intercept_:.4f}")
```

Result

自作の勾配降下: a=0.9728, b=0.0000
sklearn の fit: a=0.9728, b=0.0000

- 完全に一致。fit() の中身を、初めて自分の手で再現できた
- 元の単位に戻すと「体長 1cm あたり約 91g」という意味のある数字になる

① ノート

ブラックボックスがまた1つ開いた。分類モデルの fit() も同じ要領で開けられる（次回）。

10

モデルを複雑にしてみる

11

新しい仕入れ先の魚は15匹しかない

- 別の仕入れ先から来た魚は、まだ **15匹** しか測れていない
- 直線だけでなく、**曲線** も引ける：多項式回帰

予測 = $w_1x + w_2x^2 + \dots + w_dx^d + b$

- $x^2, x^3, \dots$ を新しい特徴量として作れば、**道具は線形回帰のまま**
- 次数 d がモデルの複雑さを決める

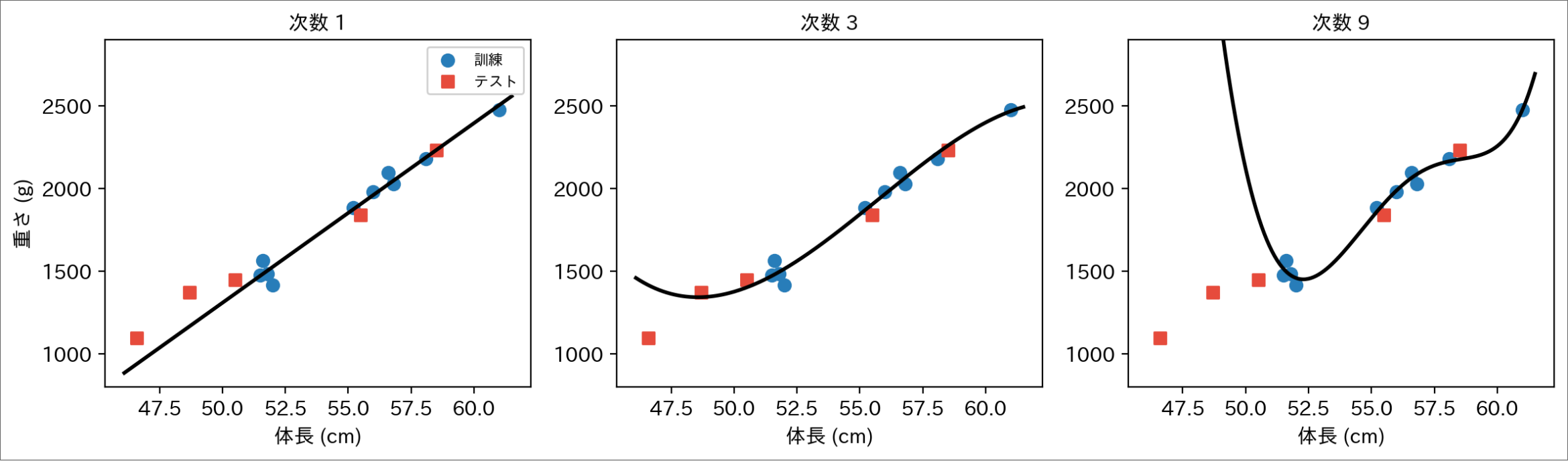
① クイズ

次数を $1 \rightarrow 3 \rightarrow 9$ と上げていくと、**訓練誤差** と **テスト誤差** はそれぞれどうなる？

12

次数を上げてみる

▶ code

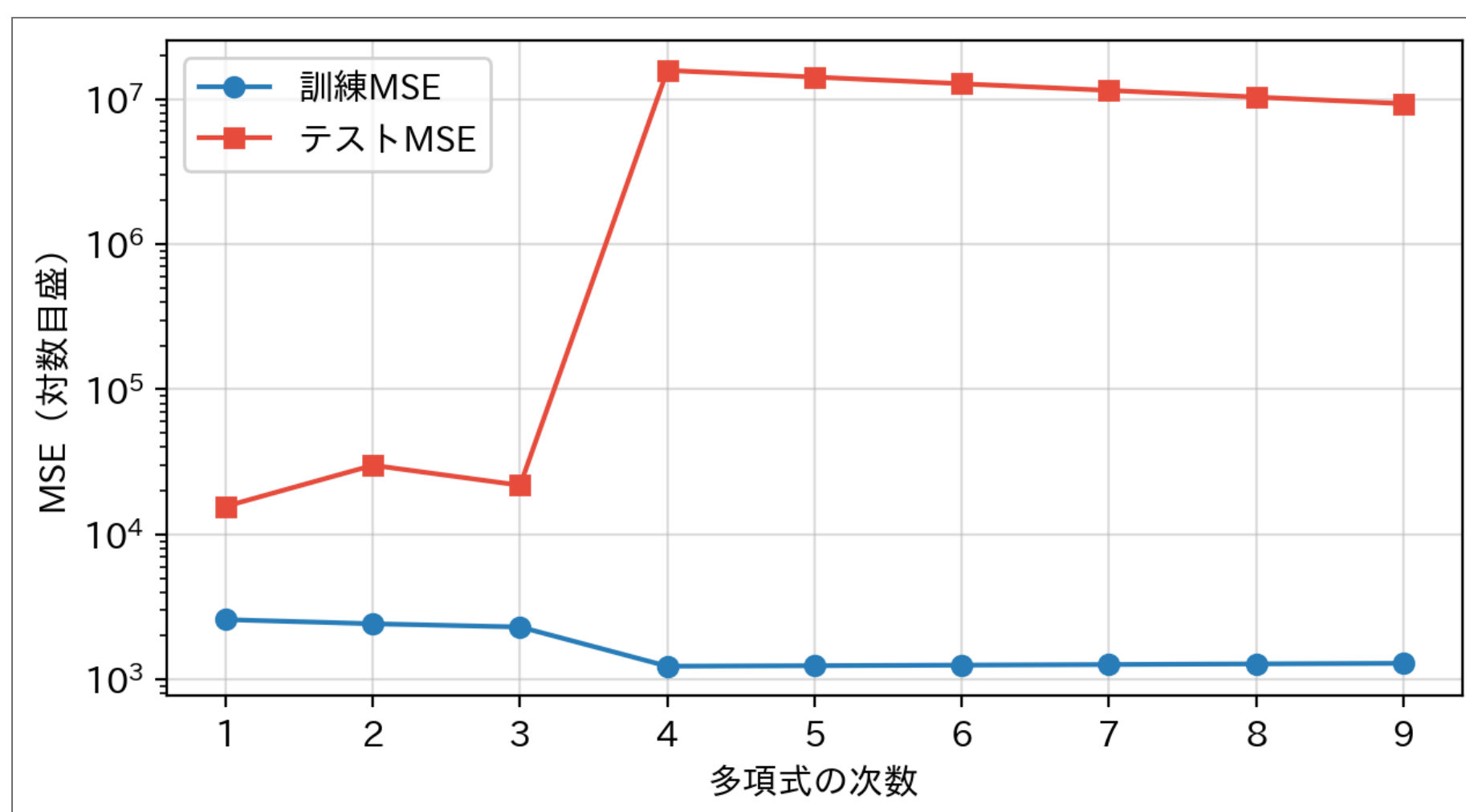


- 次数9は訓練の点を **ほぼ完璧に貫く**。代わりに左端で大暴れ — 「46.7cm の魚は 2900g 超」と主張している

数字で見る：また会ったな、過学習

Code

```
1 degrees = range(1, 10)
2 train_mse, test_mse = [], []
3 for deg in degrees:
4     model = make_pipeline(PolynomialFeatures(deg), StandardScaler(), LinearRegression())
5     model.fit(X_train, y_train)
6     train_mse.append(mean_squared_error(y_train, model.predict(X_train)))
7     test_mse.append(mean_squared_error(y_test, model.predict(X_test)))
8
9 plt.figure(figsize=(6.5, 3.6))
10 plt.semilogy(degrees, train_mse, "o-", color="#2980b9", label="訓練MSE")
11 plt.semilogy(degrees, test_mse, "s-", color="#e74c3c", label="テストMSE")
12 plt.xlabel("多項式の次数"); plt.ylabel("MSE (対数目盛)")
13 plt.legend(); plt.grid(True, alpha=0.4)
14 plt.tight_layout()
15 plt.show()
```



- 訓練誤差は下がり続け、テスト誤差は**桁違いに爆発**（縦軸は対数）
- 決定木の深さで見た過学習が、回帰でもまったく同じ形で現れた
- 深さを制限したように、複雑さを抑える道具が要る — 回帰では**正則化**

正則化：大きい係数に罰金を払わせる

- 暴れている曲線の正体は、 **巨大な係数**

Code

```
1 plain = make_pipeline(PolynomialFeatures(9), StandardScaler(), LinearRegression())
2 plain.fit(X_train, y_train)
3 print("次数9の係数の最大絶対値:", f"{np.abs(plain[-1].coef_).max():.0f}")
```

Result

次数9の係数の最大絶対値: 204,382

- そこで、損失に「係数の大きさ」への **罰金** を足す：

$$L = \text{MSE} + \alpha \sum_j w_j^2$$

- これが **Ridge 回帰**（L2正則化）
- 訓練データに合わせたい力と、係数を小さく保ちたい力の **綱引き**。 α がその強さ

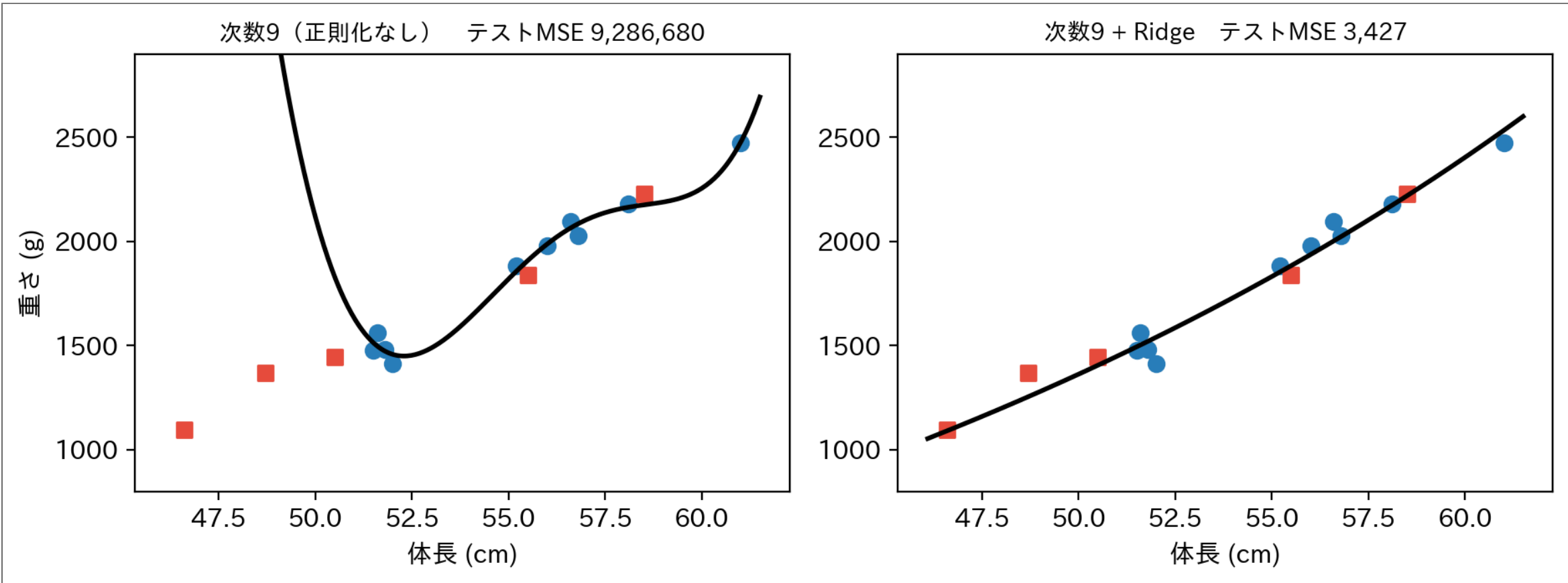
Ridge で手なずける

Code

```
1 from sklearn.linear_model import Ridge
2
3 fig, axes = plt.subplots(1, 2, figsize=(9.5, 3.6))
4 for ax, (model, name) in zip(axes, [
5     (make_pipeline(PolynomialFeatures(9), StandardScaler(), LinearRegression()),
6      "次数9 (正則化なし) "),
7     (make_pipeline(PolynomialFeatures(9), StandardScaler(), Ridge(alpha=1.0)),
8      "次数9 + Ridge")]):
9     model.fit(X_train, y_train)
10    mse = mean_squared_error(y_test, model.predict(X_test))
11    ax.scatter(X_train, y_train, c="#2980b9", s=45)
12    ax.scatter(X_test, y_test, c="#e74c3c", marker="s", s=45)
13    ax.plot(grid, model.predict(grid), "k-", lw=2)
14    ax.set_ylim(800, 2900)
15    ax.set_title(f"{name} テストMSE {mse:,.0f}", fontsize=10)
16    ax.set_xlabel("体長 (cm)")
17    print(f"{name}: 係数の最大絶対値 = {np.abs(model[-1].coef_).max():,.0f}")
18 axes[0].set_ylabel("重さ (g)")
19 plt.tight_layout()
20 plt.show()
```

Result

次数9 (正則化なし) : 係数の最大絶対値 = 204,382
次数9 + Ridge: 係数の最大絶対値 = 74



- 同じ次数9なのに、罰金を入れただけで素直な曲線に戻った
- テストMSEは**3桁以上**改善

① 過学習スレッド、完結

引き（決定木のあやしい満点）→ 正体（訓練とテストのギャップ）→ **対抗手段（複雑さの制御と正則化）**。NN でも同じ道具（L2正則化＝weight decay）がそのまま使われる。



- さっきのコード の `Ridge(alpha=1.0)` を書き換えて実行

1 罰金をほぼゼロにする

`Ridge(alpha=1e-6)`

2 罰金を強烈にする

`Ridge(alpha=1000)`

💡 ここで観察

- **1e-6** : 罰金がほぼゼロなのに、左端の大暴れは**ほぼ消える** (係数の最大絶対値 20万 → 1万) — あの暴走は、ごくわずかな罰金にすら耐えられない不安定な解だった
- **1000** : 曲線はほぼ**まっすぐ寝る** — 罰金が強すぎて、訓練データに合わせる力が負けた
- ちょうど良い α は問題ごとに違う。訓練データではなく**テストデータの MSE** で選ぶ

17

まとめ

- **回帰** : 数値を予測する。損失は**二乗誤差 (MSE)** — 微分しやすさで選ぶ
- 勾配降下法で fit を自作 → **sklearn** と**完全一致**
- **多項式回帰** : 特徴量に $x^2, x^3, \dots$ を足せば道具は線形回帰のまま
- **過学習は回帰でも起きる** : 次数を上げると訓練誤差↓・テスト誤差爆発
- **正則化 (ridge)** : 大きい係数に罰金 → 暴れる曲線を手なずける

📄 次回予告

回帰の **fit()** は開けた。次は**分類**の **fit()** — ロジスティック回帰を勾配降下法で開けると、そこに**ニューラルネットワークの最小単位**が現れる。

18

練習問題

以下の問題を **手計算** と **Pythonコード** の両方で解け。

問題1：魚2匹だけの世界で fit する

データは2点だけ： $(x_1, t_1) = (-1, 0)$ 、 $(x_2, t_2) = (1, 2)$ 。直線 $ax + b$ を勾配降下法で fit する。初期値 $a = 0, b = 0$ 、学習率 $\eta = 0.1$ 。

1. 勾配の式に代入して、 $\frac{\partial L}{\partial a}$ と $\frac{\partial L}{\partial b}$ を求めよ
2. 更新を3回、手計算で示せ
3. 同じ更新をコードで100回繰り返し、**sklearn** の答えと比較せよ

ヒント：2点を通る直線は $y = x + 1$ 。つまり答えは $a = 1, b = 1$ になるはず。

問題1の解答例

準備 ($a = 0, b = 0$ での勾配)

予測は2点とも 0。差（予測 - 実際）は $(0 - 0, 0 - 2) = (0, -2)$:

$$\frac{\partial L}{\partial a} = \frac{2}{2}[0 \times (-1) + (-2) \times 1] = -2$$

$$\frac{\partial L}{\partial b} = \frac{2}{2}[0 + (-2)] = -2$$

反復1

$$a = 0 - 0.1 \times (-2) = 0.2, \quad b = 0 - 0.1 \times (-2) = 0.2$$

反復2

差は $(b - a, a + b - 2) = (0, -1.6)$:

$$\frac{\partial L}{\partial a} = \frac{2}{2}[0 + (-1.6)] = -1.6, \quad a = 0.2 + 0.16 = 0.36$$

b もまったく同じ計算で $b = 0.36$ 。

反復3

差は $(0, -1.28)$ 、勾配はどちらも -1.28 :

$$a = 0.36 + 0.128 = 0.488, \quad b = 0.488$$

a と b が揃って 1 に近づいていく。

コードで確認

▶ 解答例を見る

Result

```
更新   1回 | a=0.2000 b=0.2000
更新   2回 | a=0.3600 b=0.3600
更新   3回 | a=0.4880 b=0.4880
更新 100回 | a=1.0000 b=1.0000
```

```
sklearn の fit: a=1.0000, b=1.0000
```

手計算の3回 ($0.2 \rightarrow 0.36 \rightarrow 0.488$) とコードの出力が一致し、100回で $a = b = 1$ (**sklearn** と同じ) に収束する。